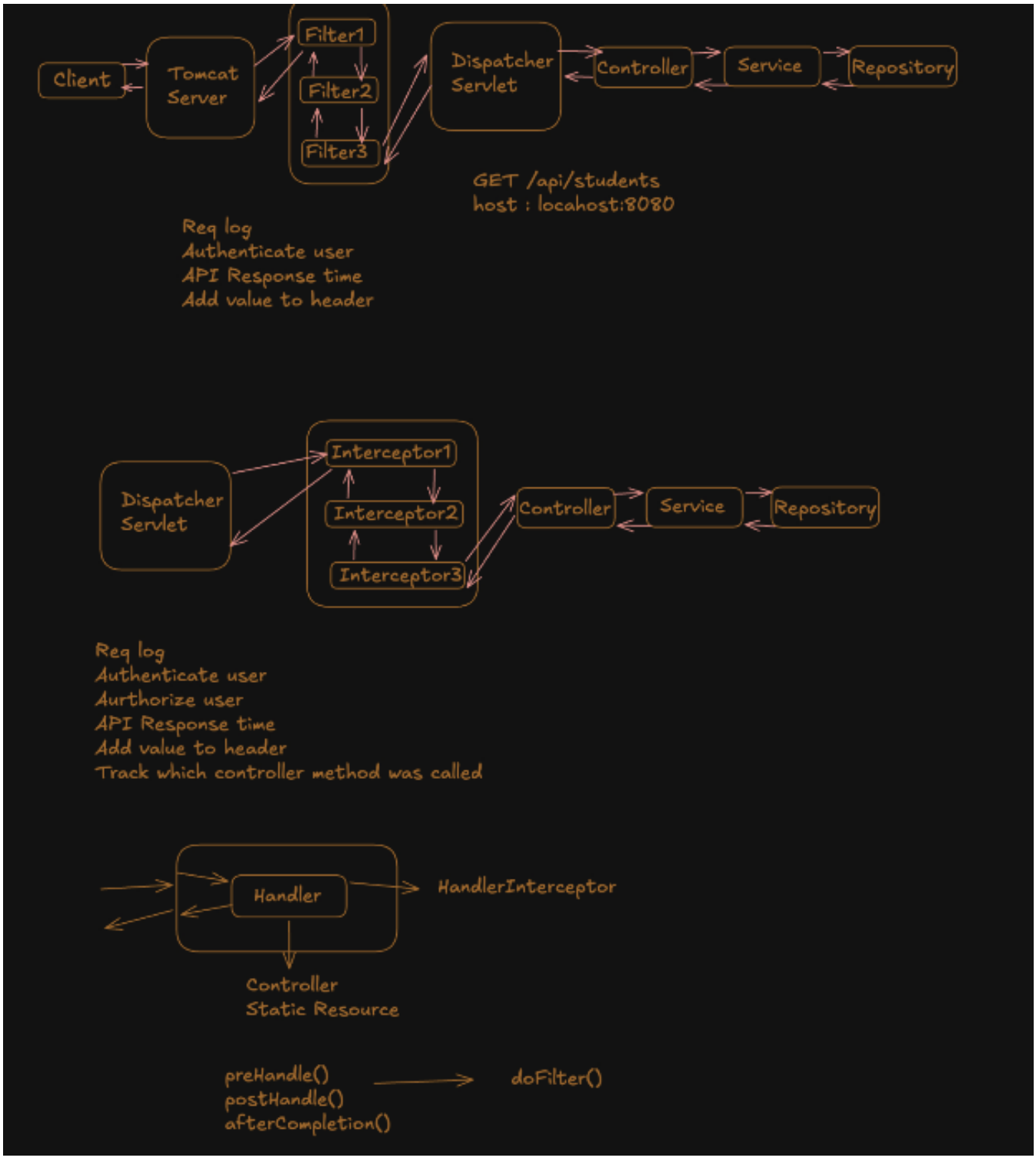
Spring Boot Interceptors

Lecture-based notes with the supplied diagrams preserved as visual references, followed by explanations of each part.

1. Filter vs Interceptor: Where Each One Runs

The first diagram compares the request path through servlet Filters with the request path through Spring MVC Interceptors. Filters run before the DispatcherServlet. Interceptors run after the DispatcherServlet has identified the handler, but before the controller handler method is invoked.



How to read the upper flow (Filters)

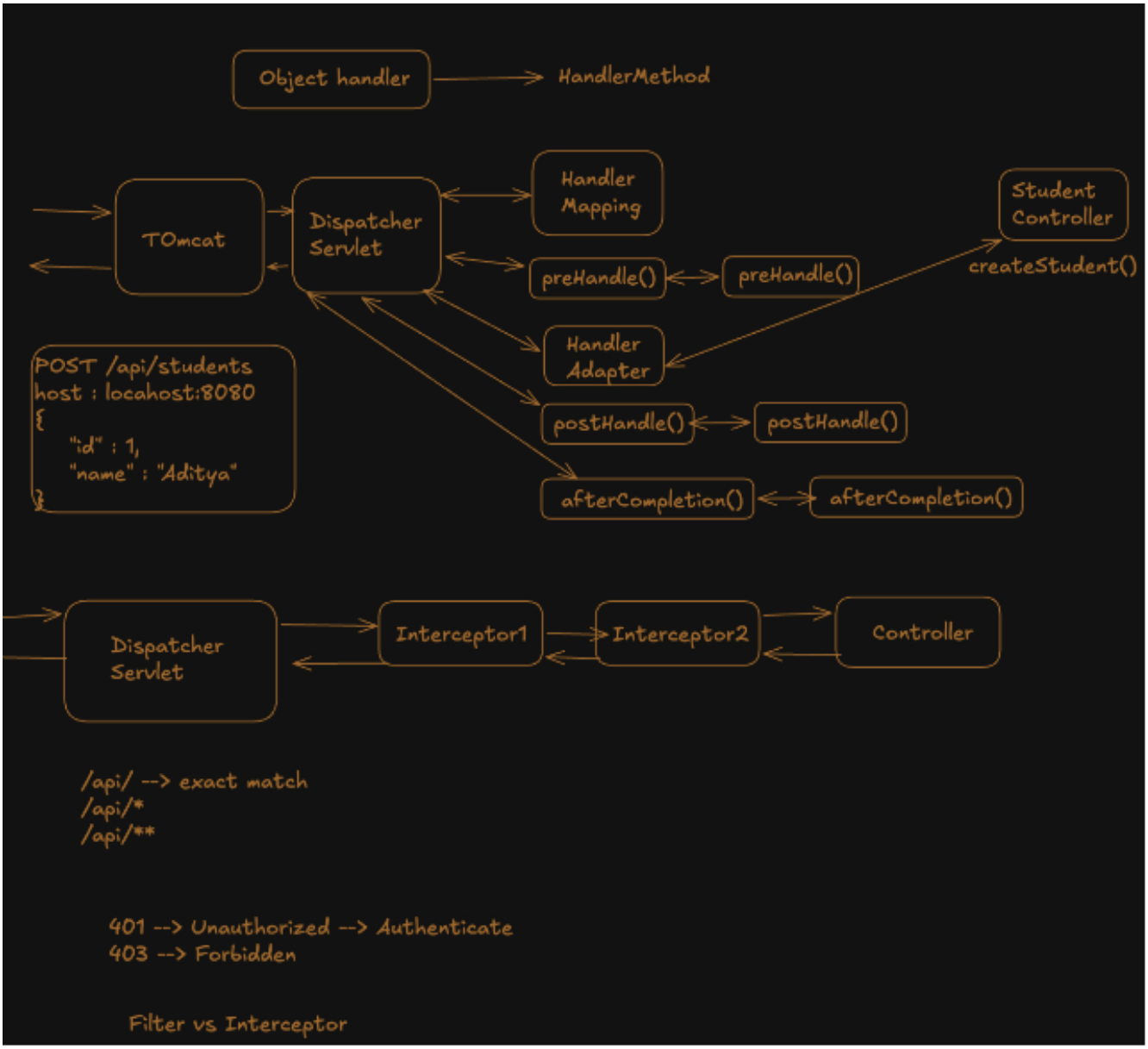
Client → Tomcat server → Filter 1 → Filter 2 → Filter 3 → DispatcherServlet → Controller → Service → Repository. The response travels back through the application path and filter chain. Filters are servlet-level components and generally see the raw HTTP request before Spring MVC selects a controller method.

Typical filter responsibilities shown in the diagram

- Request logging
- User authentication
- API response-time measurement across the broader request path
- Adding a response header

2. Interceptor Flow and Handler Lifecycle

The lower flow in the first diagram shows multiple interceptors surrounding the handler. The second diagram adds the HandlerMapping and HandlerAdapter steps, plus an example POST request.



3. The Three HandlerInterceptor Methods

preHandle() — Runs before the selected handler method. Use it for checks or logging that must happen before controller execution. Returning **true** allows processing to continue; returning **false** stops the normal handler chain.

postHandle() — Runs after the handler method has completed, before view rendering in the traditional Spring MVC view flow. It receives a ModelAndView, which may be null for REST-style responses.

afterCompletion() — Runs after request processing is complete, typically used for cleanup, final logging, timing, or inspecting an exception. The exception argument may be null.

4. DispatcherServlet, HandlerMapping, and HandlerAdapter

The lecture's request example is a POST to **/api/students** on **localhost:8080**, with a JSON body containing an id and name. The DispatcherServlet coordinates mapping and invocation.

Request flow (top to bottom)

```
Client sends HTTP request
↓
Tomcat receives the request
↓
DispatcherServlet receives it
↓
HandlerMapping identifies the matching handler method
↓
Interceptor preHandle() runs
↓
HandlerAdapter invokes the handler method
↓
Controller processes the request
↓
Interceptor postHandle() runs where applicable
↓
View rendering / response completion
↓
Interceptor afterCompletion() runs
```

HandlerMapping identifies which handler corresponds to the request. HandlerAdapter is responsible for invoking the selected handler. The interceptor receives the selected handler as an argument, which is why it can inspect which controller method is about to run.

5. Understanding the Handler Object

Spring MVC uses the generic term **handler** because a handler is not always a REST controller. Depending on the application, it may represent a controller method or another kind of resource handler. For annotation-based controller methods, the handler is commonly a HandlerMethod.

In the lecture, the handler can be checked before casting, then inspected to obtain the controller class and method name:

```
if (handler instanceof HandlerMethod handlerMethod) {
  String controllerName = handlerMethod.getBeanType().getName();
  String methodName = handlerMethod.getMethod().getName();
}
```

The instanceof check avoids blindly casting a handler that may be a different type.

6. Registering an Interceptor

Creating a class that implements HandlerInterceptor is not enough by itself. The interceptor must be registered with Spring MVC so the framework knows where to apply it.

Registration flow (top to bottom)

```
Create interceptor class
↓
Implement HandlerInterceptor
↓
Make it available as a Spring bean (for example, @Component)
↓
Create a configuration class implementing WebMvcConfigurer
↓
Override addInterceptors(InterceptorRegistry registry)
↓
Call registry.addInterceptor(yourInterceptor)
↓
Spring MVC applies the registered interceptor
```

7. Filter vs Interceptor — Quick Comparison

Aspect	Filter	Interceptor
Technology	Servlet API	Spring MVC
Position	Before DispatcherServlet	After handler mapping, before handler invocation
Knows selected controller method?	Generally no	Yes, for mapped handler methods
Common use	Broad request/response concerns	Handler-aware checks, logging, timing
Lifecycle API	init(), doFilter(), destroy()	preHandle(), postHandle(), afterCompletion()

8. URL Patterns and Status Codes Mentioned

/api/ — exact path pattern example
/api/* — matches one path segment after /api/ according to servlet-style pattern rules
/api/** — commonly used in Spring path patterns for multiple nested path segments; exact matching semantics depend on the configuration/API involved.

401 Unauthorized — typically means authentication is missing or invalid.
403 Forbidden — typically means the request is understood but access is not allowed.

9. Interview Questions

Q1. Why use an interceptor if filters already exist?
A. An interceptor runs after Spring MVC has mapped the request to a handler, so it can apply logic with knowledge of the selected controller method.

Q2. What does preHandle() return?
A. A boolean. Returning true continues processing; returning false prevents the normal handler invocation.

Q3. What is the difference between postHandle() and afterCompletion()?
A. postHandle() runs after the handler method and before view rendering in the traditional MVC flow. afterCompletion() runs after request processing is complete.

Q4. Why check handler instanceof HandlerMethod?
A. Not every handler is necessarily a controller method, so checking the type avoids an unsafe cast.

Q5. How do you register an interceptor?

A. Implement WebMvcConfigurer and register it through addInterceptors(InterceptorRegistry).